Phase 1: Core Machine Learning (Weeks 1–4)

Goal: Understand and implement classical ML algorithms with Python and Scikit-Learn.

Topics

1. **Machine Learning Basics**
   - What is ML? Types: supervised, unsupervised, reinforcement.
   - Data preprocessing: handling missing data, normalization, encoding.
2. **Supervised Learning**
   - Linear & Logistic Regression
   - Decision Trees, Random Forests
   - k-Nearest Neighbors, Naive Bayes
3. **Unsupervised Learning**
   - k-Means Clustering
   - Principal Component Analysis (PCA)
4. **Model Evaluation**
   - Train/test split, cross-validation
   - Metrics: accuracy, precision, recall, F1 score, ROC–AUC
5. **Practical Skills**
   - Use **Pandas** for data manipulation
   - Use **Matplotlib/Seaborn** for visualization
   - Learn **Scikit-Learn's API** (fit, predict, transform)

📚 **Resources**

- **Book:** *Hands-On Machine Learning with Scikit-Learn, Keras, and TensorFlow* (Aurélien Géron)
- **Course:** Kaggle's Intro to Machine Learning
- **Practice:** UCI Machine Learning Repository

🧠 **Mini-Projects**

- Predict house prices (regression)
- Classify iris flowers (classification)
- Cluster customers (unsupervised)

⚙️ **Phase 2: Deep Learning (Weeks 5–8)**

Goal: Understand and build simple neural networks using TensorFlow or PyTorch.

Topics

1. **Neural Network Fundamentals**
   - Perceptrons, activation functions, loss functions
   - Gradient descent, backpropagation
2. **Deep Learning Frameworks**
   - TensorFlow / Keras basics
   - Building and training models
3. **Computer Vision**
   - Convolutional Neural Networks (CNNs)

    - Image classification (MNIST, CIFAR-10)
4. **NLP Basics**
    - Text vectorization (Bag of Words, TF-IDF)
    - Sentiment analysis using simple networks

📚 **Resources**

- *Deep Learning Specialization* (Andrew Ng, Coursera)
- *PyTorch for Deep Learning* (freeCodeCamp YouTube)
- *Dive Into Deep Learning* (free online book)

🧠 **Mini-Projects**

- Handwritten digit recognition (MNIST)
- Sentiment analysis on tweets
- Image classifier (cats vs. dogs)

🌍 **Phase 3: Real-World ML & Deployment (Weeks 9–12)**

Goal: Learn to handle full ML pipelines, tuning, and deployment.

Topics

1. **Model Optimization**
    - Hyperparameter tuning (GridSearchCV, RandomSearch)
    - Regularization, dropout, early stopping
2. **Pipelines**
    - Scikit-Learn pipelines
    - Data preprocessing automation
3. **Deployment**
    - Save models (joblib/pickle)
    - Deploy using Flask/FastAPI
    - Basic API integration
4. **MLOps Introduction**
    - Version control for data/models
    - Model monitoring basics

📚 **Resources**

- *Made With ML* (madewithml.com)
- *Full Stack Deep Learning* (YouTube)

🧠 **Capstone Project**

Choose one:

- Predict stock price movements
- Build a movie recommender
- Deploy a web app for image or text classification

Machine Learning

It is a branch of **artificial intelligence (AI)** that focuses on **teaching computers to learn from data and improve automatically** without being explicitly programmed for every task.
Instead of writing rules by hand, we **feed data** to an algorithm, and the algorithm **finds patterns** in that data to make **predictions or decisions**.

In simple terms

Traditional programming	Machine Learning
You give the computer **rules** + **data** → it produces an **answer**.	You give the computer **data** + **answers** → it learns the **rules**.

How Machine Learning Works (at a high level)

1. **Collect Data**
   (e.g., customer info, images, text)
2. **Prepare Data**
   Clean, normalize, and split into training/testing sets.
3. **Choose a Model**
   (e.g., linear regression, decision tree, neural network)
4. **Train the Model**
   Feed it data so it learns patterns.
5. **Test the Model**
   Evaluate accuracy on new (unseen) data.
6. **Use the Model**
   Make predictions or decisions automatically.

Main Types of Machine Learning

Type	What it does	Example
Supervised Learning	Learns from labeled data (has right answers).	Predict house prices, classify emails.
Unsupervised Learning	Finds patterns in unlabeled data.	Customer segmentation, topic grouping.
Reinforcement Learning	Learns by trial and error using rewards.	Training robots, self-driving cars, game AI.

Data Preprocessing?

Data preprocessing means **cleaning, transforming, and organizing raw data** so that it can be efficiently used by a machine learning model.

Raw data usually has:

- Missing values
- Duplicates
- Inconsistent formats

- Irrelevant or noisy features
- Different scales (e.g., prices vs. ages) > Normalization

We fix these issues before training the model.

Steps in Data Preprocessing

1. **Collect the Data**
   CSV files, Databases (SQL), APIs, Web scraping, public sources (like Kaggle or UCI Repository)
2. **Inspect the Data**
3. **Handle Missing Data**
   Remove Missing Values,
   Fill (Impute) Missing Values
4. **Handle Categorical (Text) Data**
   Label Encoding, One-Hot Encoding
5. **Feature Scaling**
   Standardization (Z-score), Normalization (Min-Max Scaling)
6. **Split Data**
   Training set (e.g., 80%) → model learns here,
   Testing set (e.g., 20%) → model is evaluated here
7. **Feature Selection (Optional)**

Supervised Learning?

Supervised learning means we **train a model on labelled data** — data where we already know the correct answers (called labels or targets).

The goal is for the model to learn the relationship between input features (X) and output labels (y), so it can predict labels for **new**, **unseen** data.

Let's say you have data about houses:

Size (m^2)	**Bedrooms**	**Price ($)**
50	1	150,000
80	2	220,000
120	3	310,000

Here:

- **Features (X):** Size, Bedrooms
- **Target (y):** Price

You can train a supervised model to predict house prices for new houses.

Main Types of Supervised Learning

Type	Output	Example
Regression	Predicts continuous values	Predict house prices, stock values
Classification	Predicts categories (labels)	Spam or not spam, disease or no disease

Linear Regression (Classification or Regression)

Predict a continuous number. It finds the *best-fitting straight line* through your data.

$$y = w_1 \times x_1 + w_2 \times x_2 + \ldots + b$$

```
from sklearn.linear_model import LinearRegression
model = LinearRegression()
model.fit(X_train, y_train)
y_pred = model.predict(X_test)
```

Logistic Regression (Classification or Regression)

Despite the name, this is for **classification**, not regression. It predicts probabilities using the **sigmoid function**:

$$P\left(y = \frac{1}{x}\right) = \frac{1}{1 + e^{-(wx+b)}}$$

If probability > 0.5 ⟶ class = 1 (e.g., "spam")

Else ⟶ class = 0 (e.g., "not spam")

```
from sklearn.linear_model import LogisticRegression
model = LogisticRegression()
model.fit(X_train, y_train)
y_pred = model.predict(X_test)
```

Decision Trees (Classification or Regression)

A **tree-like structure** that splits data based on feature values (like 20 questions). Each node makes a decision ⟶ until it reaches a final prediction.

Advantages:

- Easy to visualize
- Handles numerical + categorical data
- No need for feature scaling

```
from sklearn.tree import DecisionTreeClassifier
model = DecisionTreeClassifier(max_depth=3)
model.fit(X_train, y_train)
y_pred = model.predict(X_test)
```

Random Forests (Classification or Regression)

An **ensemble method** — combines many decision trees. Each tree votes, and the forest takes the majority vote (for classification) or average (for regression).

Advantages:

- More accurate than a single tree
- Handles large datasets well
- Resistant to overfitting

```
from sklearn.ensemble import RandomForestClassifier
model = RandomForestClassifier(n_estimators=100, random_state=42)
model.fit(X_train, y_train)
y_pred = model.predict(X_test)
```

k-Nearest Neighbors (kNN) (Classification or Regression)

Makes predictions based on the **k closest data points** in the training set. If most neighbors are class "A," the new point is also "A."

Key idea: "Show me your neighbors, I'll tell you who you are."

```
from sklearn.neighbors import KNeighborsClassifier
model = KNeighborsClassifier(n_neighbors=5)
model.fit(X_train, y_train)
y_pred = model.predict(X_test)
```

Naive Bayes (Classification)

Based on **Bayes' Theorem**, calculate probabilities for each class given feature values. "Naive" because it assumes features are independent (which isn't always true, but works well in practice).

Great for: Text classification (spam filters, sentiment analysis)

```
from sklearn.naive_bayes import GaussianNB
model = GaussianNB()
model.fit(X_train, y_train)
y_pred = model.predict(X_test)
```

Model Evaluation Metrics

Task	Metrics
Regression	Mean Squared Error (MSE), R^2 score
Classification	Accuracy, Precision, Recall, F1-score, Confusion Matrix

Summary

```
from sklearn.metrics import accuracy_score, classification_report
print("Accuracy:", accuracy_score(y_test, y_pred))
print(classification_report(y_test, y_pred))
```

Algorithm	Type	Strengths	Weaknesses
Linear Regression	Regression	Simple, fast	Only works with linear data
Logistic Regression	Classification	Probabilistic	Can underfit non-linear data
Decision Tree	Both	Interpretable	Can overfit
Random Forest	Both	High accuracy	Less interpretable
kNN	Both	Simple	Slow on large data
Naive Bayes	Classification	Great for text	Strong independence assumption

Unsupervised Learning

It's deals with **unlabeled data** — the algorithm tries to **find hidden patterns, structures, or groupings** in the data *without knowing the correct answers in advance.*

What Is Unsupervised Learning?

- In supervised learning, you teach the model "this is a cat, this is a dog."
- In unsupervised learning, you just give it a bunch of images, and it tries to **group similar ones together**.

Main Types of Unsupervised Learning

Type	Goal	Example
Clustering	Group similar data points	Customer segmentation, document grouping
Dimensionality Reduction	Simplify high-dimensional data while keeping important info	Image compression, visualization

k-Means Clustering (Clustering)

What It Does

k-Means groups your data into **k clusters** (you choose k).

Each cluster has a **centroid** (the "center" of that group).

The algorithm:

1. Chooses k random points as initial centroids.
2. Assigns each data point to its nearest centroid.
3. Recalculates centroids.
4. Repeats until cluster assignments stop changing.

Imagine you have customer data:

Age	Annual Income ($)
22	25,000
25	30,000
40	60,000

45	65,000

The algorithm might find two clusters:

- Cluster 1 → Young, low income
- Cluster 2 → Older, higher income

```
from sklearn.cluster import KMeans
import matplotlib.pyplot as plt

# Example data
X = [[22, 25000], [25, 30000], [40, 60000], [45, 65000]]

# Create model
kmeans = KMeans(n_clusters=2, random_state=42)
kmeans.fit(X)

# Get cluster labels and centers
labels = kmeans.labels_
centroids = kmeans.cluster_centers_

print("Labels:", labels)
print("Centroids:", centroids)

# Plot clusters
for i in range(len(X)):
    plt.scatter(X[i][0], X[i][1], c='red' if labels[i]==0 else 'blue')
plt.scatter(centroids[:,0], centroids[:,1], c='black', marker='x')
plt.xlabel("Age")
plt.ylabel("Income")
plt.title("k-Means Clustering")
plt.show()
```

How to Choose k?

Use the **Elbow Method** — plot the "inertia" (sum of distances to cluster centers) vs. k, and look for where the curve bends (the "elbow").

```
inertias = []
for k in range(1, 10):
    kmeans = KMeans(n_clusters=k)
    kmeans.fit(X)
    inertias.append(kmeans.inertia_)

plt.plot(range(1,10), inertias, marker='o')
plt.xlabel('Number of Clusters (k)')
plt.ylabel('Inertia')
plt.show()
```

Principal Component Analysis (PCA) (Dimensionality Reduction)

What It Does

PCA helps simplify data by **reducing the number of features (dimensions)** while keeping the most important patterns (variance).

It finds **new "principal components"** — directions in your data that explain the most variance.

In short:

From 100 features → maybe just 2 or 3 "summary" features that still capture most of the information.

Why It's Useful

- Visualize high-dimensional data in 2D/3D.
- Speed up ML models (less computation).
- Remove noise and redundancy.

Example

Imagine an image has 1,000 pixels (features).

PCA might find that only 50 of them capture 95% of the variance.

So you can work with 50 features instead of 1,000.

```
from sklearn.decomposition import PCA
from sklearn.datasets import load_iris
import matplotlib.pyplot as plt

# Load sample dataset
data = load_iris()
X = data.data

# Reduce dimensions from 4D → 2D
pca = PCA(n_components=2)
X_pca = pca.fit_transform(X)

# Plot results
plt.scatter(X_pca[:,0], X_pca[:,1], c=data.target)
plt.xlabel("Principal Component 1")
plt.ylabel("Principal Component 2")
plt.title("PCA on Iris Dataset")
plt.show()

# Check explained variance
print("Explained variance ratio:", pca.explained_variance_ratio_)
```

Explained Variance

Tells you how much information (variance) is kept:

- Component 1: 70%
- Component 2: 20%
- Total: 90% → means you retained 90% of the info.

Summary

Algorithm	Type	Goal	Example Use
k-Means	Clustering	Group similar items	Customer segmentation
PCA	Dimensionality Reduction	Compress/simplify data	Visualization, noise removal

Key Differences

Aspect	k-Means	PCA
Goal	Group data points	Reduce dimensions
Input	Data points	Data points (high-dim)
Output	Cluster labels	Transformed data (lower-dim)
Use Case	Segmentation	Visualization, compression

What Is Model Evaluation?

Model Evaluation tells you **how well your ML model performs** — not just on the data it learned from (training data), but more importantly, on **new, unseen data** (testing data).
The goal is to ensure your model **generalizes** — it doesn't just memorize, but actually learns patterns.

Train/Test Split

When you train a model, you must **separate your data** into at least two parts:

Dataset	Purpose
Training Set	Used to train the model (find patterns)
Testing Set	Used to evaluate how well the model performs on unseen data

```
from sklearn.model_selection import train_test_split
from sklearn.datasets import load_iris

# Load data
X, y = load_iris(return_X_y=True)

# Split: 80% training, 20% testing
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)
```

Now, you train the model on X_train, y_train, and test on X_test, y_test.

Cross-Validation

Sometimes, splitting once isn't enough — performance might vary depending on how the data is split.
To get a **more reliable estimate**, we use **k-fold cross-validation**.

How It Works

1. Split data into **k equal parts** (folds)
2. Train on (k−1) folds, test on the remaining one
3. Repeat for all folds
4. Average the performance scores

```
from sklearn.model_selection import cross_val_score
from sklearn.ensemble import RandomForestClassifier

model = RandomForestClassifier()
scores = cross_val_score(model, X, y, cv=5)  # 5-fold CV
print("Cross-validation scores:", scores)
print("Mean accuracy:", scores.mean())
```

Model Evaluation Metrics

Different types of ML tasks need different evaluation metrics.

Accuracy For Classification

Fraction of correct predictions:

$$Accuracy = \frac{TP+TN}{TP+TN+FP+FN}$$

- **TP:** True Positive (correctly predicted "yes")
- **TN:** True Negative (correctly predicted "no")
- **FP:** False Positive (incorrectly predicted "yes")
- **FN:** False Negative (missed a "yes")

Works well when classes are balanced.

Misleading for **imbalanced data** (e.g., 99% non-fraud, 1% fraud).

Precision

"How many predicted positives were actually positive?"

$$Precision = \frac{TP}{TP+FP}$$

High precision = few false alarms.

Recall (Sensitivity)

"How many actual positives did we correctly predict?"

$$Recall = \frac{TP}{TP+FN}$$

High recall = you catch most of the positive cases.

F1 Score

Harmonic mean of precision and recall:

$$F1 = 2\times\left(\frac{Precision\times Recall}{Precision+Recall}\right)$$

Useful when you want a balance between **precision and recall**.

ROC–AUC (Receiver Operating Characteristic – Area Under Curve)

Measures the model's ability to **separate classes**.

- **ROC curve:** Plots True Positive Rate (Recall) vs. False Positive Rate.
- **AUC (Area Under Curve):**
  - 1.0 → perfect classifier
  - 0.5 → random guessing

```
from sklearn.metrics import (
    accuracy_score,
    precision_score,
    recall_score,
    f1_score,
    roc_auc_score,
    confusion_matrix
)

# Example predictions
y_true = [0, 1, 1, 0, 1, 0, 1]
y_pred = [0, 1, 0, 0, 1, 0, 1]

print("Accuracy:", accuracy_score(y_true, y_pred))
print("Precision:", precision_score(y_true, y_pred))
print("Recall:", recall_score(y_true, y_pred))
print("F1 Score:", f1_score(y_true, y_pred))

# If your model outputs probabilities:
# y_pred_proba = model.predict_proba(X_test)[:, 1]
# print("ROC-AUC:", roc_auc_score(y_test, y_pred_proba))
```

Confusion Matrix

A 2×2 table showing prediction results:

	Predicted Positive	**Predicted Negative**
Actual Positive	TP	FN
Actual Negative	FP	TN

```
from sklearn.metrics import ConfusionMatrixDisplay
ConfusionMatrixDisplay.from_predictions(y_true, y_pred)
```

For Regression

Metric	Description
MSE (Mean Squared Error)	Penalizes large errors
MAE (Mean Absolute Error)	Average of absolute errors
R^2 (R-squared)	How much variance is explained by the model

```
from sklearn.metrics import mean_squared_error, r2_score

print("MSE:", mean_squared_error(y_true, y_pred))
print("R²:", r2_score(y_true, y_pred))
```

Summary Table

Metric	Type	When to Use
Accuracy	Classification	Balanced datasets
Precision	Classification	When false positives are costly
Recall	Classification	When false negatives are costly
F1-score	Classification	When both matter
ROC–AUC	Classification	To compare models' ability to distinguish classes
MSE / MAE / R^2	Regression	Continuous targets

Quick Analogy

Model	Metric Focus	Analogy
Precision	Be sure before saying "yes"	Doctor only says "disease" if 100% sure
Recall	Catch every true case	Doctor catches all diseases, even if false alarms
F1	Balance both	Sensible trade-off
ROC–AUC	Ranking ability	How well it ranks positives above negatives

Reinforcement Learning?

It is a type of Machine Learning where an **agent** learns to make decisions by interacting with an **environment** to achieve a **goal**.

RL System Components

Component	Description
Agent	The learner or decision-maker (e.g., robot, software)
Environment	The world where the agent acts (e.g., game, road, market)
State (S)	A snapshot of the environment at a given time
Action (A)	What the agent can do in each state

Reward (R)	Feedback signal (positive or negative) after taking an action
Policy (π)	The agent's strategy — mapping states to actions
Value Function (V)	Predicts long-term reward for a state
Q-Function (Q)	Predicts reward for a state–action pair

RL Learning Cycle

1. **Agent observes** the current **state (S)**
2. **Agent takes an action (A)**
3. **Environment gives** a **reward (R)** and new **state (S')**
4. **Agent updates** its knowledge (policy/value)
5. Repeat until the agent learns the best policy

Types of Reinforcement Learning

Type	Description	Example
Positive RL	Agent receives **positive rewards** for good actions → reinforces behavior	Dog gets treat for sitting
Negative RL	Agent receives **penalty** for bad actions → discourages behavior	Car AI gets negative reward for crashing

Key RL Algorithms

1. **Dynamic Programming**
   - Solves RL problems if the environment model is known.
   - Example: *Value Iteration*, *Policy Iteration*
2. **Monte Carlo Methods**
   - Learn by averaging rewards from episodes (no model needed).
3. **Temporal Difference (TD) Learning**
   - Learn from partial experience; updates predictions as it goes.
   - Basis for **Q-Learning** and **SARSA**.
4. **Q-Learning**
   - Learns the *optimal action-value function* $Q(s, a)$.
   - Formula:

$$Q(s, a) = Q(s, a) + \alpha[R + \gamma max_{a'} Q(s', a') - Q(s, a)]$$

where:

- α: learning rate
- γ: discount factor
- R: reward
- s': next state

5. **Deep Q-Networks (DQN)**
   - Combines **Q-learning** with **deep neural networks**
   - Used in **Atari games, AlphaGo, autonomous driving**

Self-Driving Car

RL Component	Example
Agent	The car's control system
Environment	The road, traffic, signals
State	Car's speed, lane position, obstacles
Action	Accelerate, brake, turn left/right
Reward	+1 for staying in lane, -10 for crash

Over time, the car learns which actions maximize the total reward → smooth, safe driving.

Summary

Concept	Description
Goal	Maximize total cumulative reward
Learns from	Experience and feedback
No labeled data	Learns by trying and failing
Key algorithms	Q-Learning, SARSA, Deep Q-Networks
Applications	Robotics, Gaming (AlphaGo), Trading, Self-driving cars